

INSTALLATION & DEPLOYMENT GUIDE

Cyber-Physical Smart Building Access Control System

1. Components & Bring-Up Order

The system has five independently deployable pieces. Bring them up in this order — each one depends on the previous being reachable, and starting out of order just produces confusing "not connected yet" log lines rather than an actual failure:

#	Component	Depends on
1	MQTT broker (Mosquitto, HiveMQ, or similar)	Nothing — this is the hub everything else connects to
2	Backend (FastAPI + database)	The broker (to receive live status/events); the database can be SQLite with zero setup
3	Building Gateway (only if any doors use the RS-485 backbone)	The broker + the RS-485 bus being wired
4	Door node firmware (on each ESP32)	Wi-Fi + broker reachable directly, OR the Gateway if using RS-485
5	Dashboard (React SPA)	The backend's REST API

2. MQTT Broker

Any MQTT 3.1.1 broker works — Mosquitto is the common choice for a real deployment. For development/testing without installing anything system-wide, a pure-Python broker works too:

```
pip install amqtt
amqtt # binds 0.0.0.0:1883, anonymous auth allowed by default
```

Production note: enable authentication (username/password or client certificates) and TLS on the broker before deploying — the backend, gateway, and firmware all support MQTT_USE_TLS / credentials, but the broker has to actually require them.

3. Backend

```
cd backend
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # edit DATABASE_URL, JWT_SECRET, MQTT_* as needed
python -m scripts.seed_db # creates tables + demo users/doors/credentials
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Required .env values for a real deployment (the shipped defaults are dev-only placeholders):

Variable	Notes
DATABASE_URL	SQLite by default (zero setup); switch to PostgreSQL for anything beyond a pilot — see Section 7
JWT_SECRET	Long random string; rotating it invalidates all existing sessions
CREDENTIAL_ENCRYPTION_KEY /	Fernet key + HMAC key for credential encryption at rest (Phase 5). Must

Variable	Notes
CREDENTIAL_INDEX_KEY	be set to real random values and persisted — losing them makes stored card UUIDs permanently unreadable
MQTT_BROKER_HOST / PORT / USE_TLS / USERNAME / PASSWORD	Point at the broker from Section 2
DOOR_STALE_AFTER_SECONDS / DOOR_STALENESS_CHECK_INTERVAL_SECONDS	Phase 7 staleness watchdog tuning — defaults (30s / 10s) are reasonable for a single building

Visit <http://<host>:8000/docs> to confirm the API is up and browse every endpoint interactively (see `API_Reference.docx` for the full written reference).

4. Building Gateway (only if using the RS-485 backbone)

Skip this section entirely if every door node connects directly over Wi-Fi. Deploy this if doors share an RS-485 multi-drop bus (see `Phase6_Multi_Node_Deployment_Guide.docx` for the wiring/addressing details).

```
cd gateway
pip install -r requirements.txt
cp gateway_config.example.yaml gateway_config.yaml
# edit: serial.port (the USB-RS485 adapter's device path),
# nodes: (addr -> door code mapping, must match each node's config.h)
python rs485_gateway.py --config gateway_config.yaml
```

The gateway needs no changes to the backend or database — a gateway-relayed door looks identical to a directly-connected one on the MQTT topics both use.

5. Door Node Firmware (per ESP32)

```
cd door_node_firmware
cp include/secrets_example.h include/secrets.h # fill in Wi-Fi + broker credentials
# edit include/config.h for THIS specific node:
# DOOR_ID, DOOR_REQUIRES_BIO, DOOR_FAIL_MODE_SAFE
# RS485_NODE_ADDR (only relevant if this node is on the RS-485 backbone)
# DESFIRE_AES_KEY (replace the placeholder before provisioning real cards)
pio run --target upload
pio device monitor
```

Wire per `Phase2_Wiring_and_Bench_Test_Guide.docx` (single node) and, if applicable, `Phase6_Multi_Node_Deployment_Guide.docx` (RS-485 bus topology/addressing for multiple nodes). Run the Phase 2 bench test checklist on every node before it goes into a real door.

Important: this firmware has been reviewed line-by-line but never compiled/run on real ESP32 hardware in this project's development environment (no hardware, no working ESP32 toolchain available there). Treat `pio run` as your first real verification step, not this document.

6. Dashboard

```
cd dashboard
npm install
echo "VITE_API_BASE_URL=http://<backend-host>:8000" > .env
npm run build # production build -> dist/
# serve dist/ with any static file server (nginx, serve, etc.)
# OR for local development only:
npm run dev
```

The dashboard is a static SPA — it can be hosted anywhere that can serve static files and reach the backend's REST API. Set `VITE_API_BASE_URL` to wherever the backend actually runs; it defaults to `http://localhost:8000` for local development.

7. Database: SQLite vs. PostgreSQL

SQLite is the zero-setup default and was load-tested at pilot scale (16 concurrent simulated nodes, 0 dropped events — see the Phase 6 deployment guide) without issue. For anything beyond a pilot, switch DATABASE_URL to PostgreSQL:

```
DATABASE_URL=postgresql+psycpg2://user:password@host:5432/access_control
```

No code changes are needed — the backend uses SQLAlchemy throughout. Install psycpg2-binary (not in requirements.txt by default, to keep the zero-setup SQLite path dependency-free) and re-run `python -m scripts.seed_db` against the new database.

8. Post-Install Checklist

1. GET /health on the backend returns { "status": "ok" }.
2. Log into the dashboard with the seeded admin account (or a real one you've created via POST /api/users) and confirm the door list loads.
3. Power on one door node (or the gateway + its nodes) and confirm its status flips to online in the dashboard within a few seconds.
4. Present a provisioned card at that door and confirm an access event appears in that door's log.
5. Trigger the tamper switch and confirm an alert appears and the door force-relocks.
6. Change the default admin/instructor passwords and rotate JWT_SECRET / CREDENTIAL_ENCRYPTION_KEY / CREDENTIAL_INDEX_KEY away from the repository's example values before going anywhere near production data.